Sunthareswaran
Sharujan

TD03 : Projet MarketPulse

Git en local : garder un historique fiable des changements

Module MESIFI472326 – Python, Git, Linux · Groupe G07 · Équipe TSHA



Enseignant : ELANGA NKO'O Steve Patrick Lionel **Année Universitaire :2026/2027**

Table des matières

1. Introduction **3**

2. TD03 – Git Local Workflow **3**

2.1. Partie 1 – Le modèle Git local 3
2.2. Partie 2 – Identité Git 3
2.3. Partie 3 – Les changements du TD02 3
2.4. Partie 4 – git status et git diff 4
2.5. Partie 5 – Indexer le changement 4
2.6. Partie 6 – Le commit 5
2.7. Partie 7 – L'historique 6
2.8. Partie 8 – Qualité du commit 7
2.9. Partie 9 – Local et distant 7
2.10. Partie 10 – Validation 7

3. Conclusion **7**

1. Introduction

Le TD03 ouvre la partie Git du projet MarketPulse. Le besoin du desk est de garder un historique fiable des changements : savoir ce qui a changé dans le code, qui l'a changé et quand. On reste en local : on apprend à lire l'état du dépôt, à choisir ce qu'on enregistre, à créer un commit et à relire l'historique. Les branches et le push viendront aux TD suivants.

J'ai fait ce TD en autonomie avant la séance, dans mon Codespace sur le dépôt de l'équipe. Le commit créé correspond au travail Python du TD02, qui n'avait pas encore été enregistré.

2. TD03 – Git Local Workflow

2.1. Partie 1 – Le modèle Git local

Un changement passe par trois zones : le working tree (les fichiers que je modifie), la staging area (les changements choisis avec `git add`) et l'historique local (les versions enregistrées avec `git commit`). Enregistrer un fichier dans l'éditeur ne crée donc pas de commit : il faut d'abord l'ajouter à la staging area, puis commiter.

2.2. Partie 2 – Identité Git

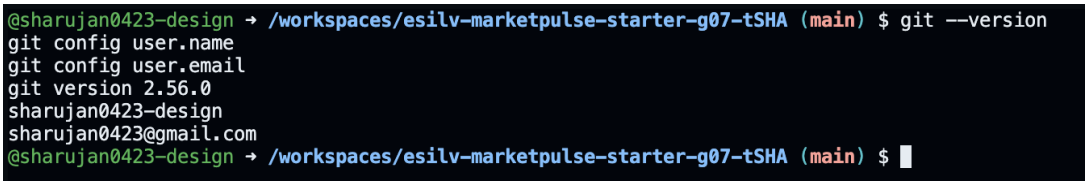


Figure 1 – Version de Git et identité configurée.

Git 2.56.0 est installé et l'identité correspond à mon compte GitHub (`sharujan0423-design`). Chaque commit sera donc attribué à mon nom.

2.3. Partie 3 – Les changements du TD02

Dans mon Codespace, je me suis placé sur `main` et j'ai remis dans `src/main.py` la version finale de mon code du TD02 :

```
$ git status --short
 M src/main.py

$ git diff --stat
 src/main.py | 45 ++++++++++++++-------
 1 file changed, 30 insertions(+), 15 deletions(-)
```

Terminal

Au premier `git diff`, des lignes vides apparaissaient comme modifiées : le copier-coller avait ajouté des espaces invisibles en fin de ligne. Je les ai supprimés avec `sed -i 's/[[:space:]]*$//' src/main.py` pour ne garder que les vrais changements. Le diff complet est visible plus bas avec `git show HEAD` (figure 8).

- Fichier modifié : seulement `src/main.py`.
- Lignes ajoutées : les fonctions `get_first_close`, `get_last_close` et `display_market_summary`, le bloc « Market configuration » et les deux appels au résumé dans `main()`.

- Lignes supprimées : l'ancien affichage du starter (« Last price », « Last level » et le bloc « Observations »).
- Le diff correspond bien au travail du TD02. La dernière ligne `main()` apparaît supprimée puis rajoutée seulement à cause du retour à la ligne final du fichier.
- Il ne reste pas de print de debug : ceux du TD02 avaient déjà été retirés.

2.4. Partie 4 – git status et git diff

`git status` donne l'état du dépôt, `git status --short` le résume en une ligne par fichier et `git diff` montre le détail des changements non indexés. Après le commit, ces commandes n'affichent plus rien à part « working tree clean » :

```
git status --short
git diff
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Figure 2 – git status --short, git diff et git status après le commit.

Dans `git status --short`, `M` (M à droite) veut dire fichier suivi modifié mais pas encore indexé, `M` (M à gauche) veut dire changement indexé, et `??` désigne un fichier non suivi, que Git ne connaît pas encore.

2.5. Partie 5 – Indexer le changement

J'ai indexé uniquement `src/main.py` avec `git add src/main.py`, sans utiliser `git add .` qui prendrait tout le dossier :

```
$ git add src/main.py
$ git status
On branch main
Changes to be committed:
        modified:   src/main.py

$ git diff --staged --stat
 src/main.py | 45 +++++++++++++-------
 1 file changed, 30 insertions(+), 15 deletions(-)
```

Terminal

Comme ces commandes étaient passées avant mes captures, j'ai refait la manipulation avec une ligne de test (`# test partie 5`) :

```
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (main) $
                                git status                    git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   src/main.py

@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (main) $
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (main) $
```

Figure 3 – git status : la modification est dans « Changes to be committed ».

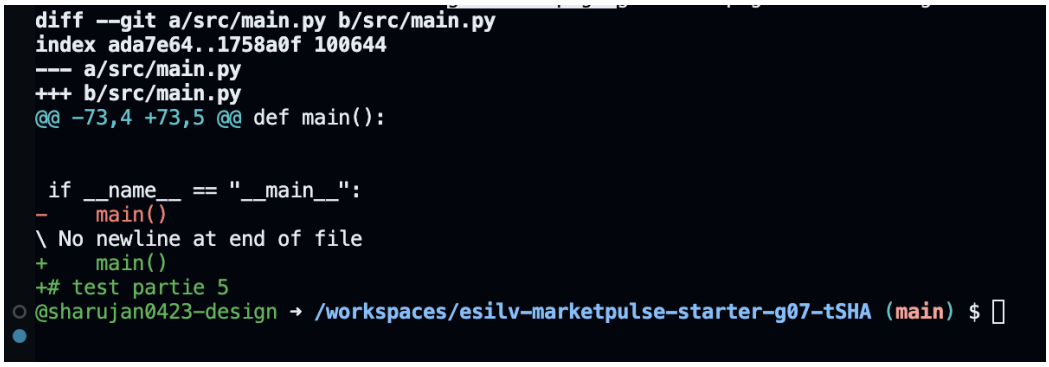


Figure 4 – git diff --staged : la ligne ajoutée apparaît en vert.

`git diff --staged` montre ce qui partira dans le commit, alors que `git diff` n'affiche plus rien puisque tout est indexé. J'ai ensuite annulé le test avec `git restore --staged` puis `git restore` :

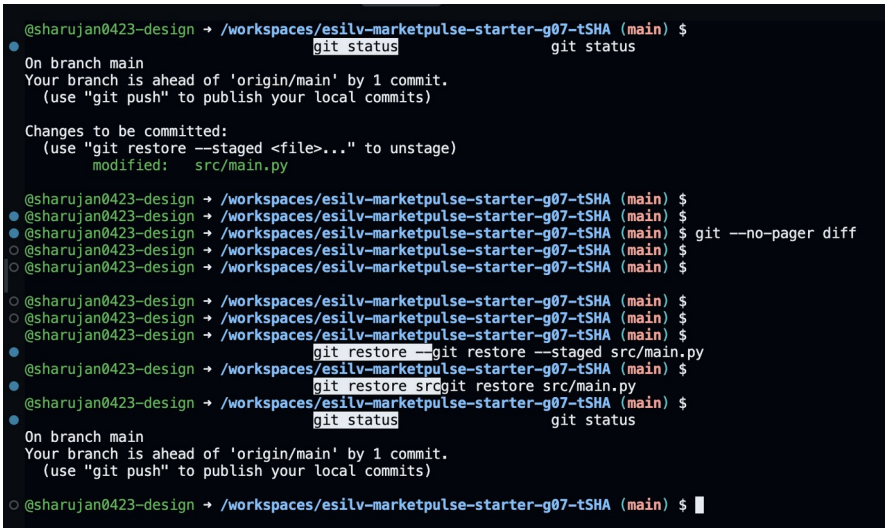


Figure 5 – git diff vide, puis annulation du test avec git restore.

Remarque : le terminal de Codespaces effaçait parfois la dernière ligne d'une sortie, ce qui explique que « nothing to commit » n'apparaisse pas sur cette capture.

2.6. Partie 6 – Le commit

Avant de commiter, j'ai vérifié que le programme fonctionnait toujours :



Figure 6 – python src/main.py avant le commit.

```
$ git commit -m "feat: add reusable market summary for instrument and benchmark"
[main 9b013ed] feat: add reusable market summary for instrument and benchmark
 1 file changed, 30 insertions(+), 15 deletions(-)
```

Terminal

Le message décrit le vrai changement, avec le préfixe `feat:` pour une nouvelle fonctionnalité, plutôt qu'un message vague comme « update » ou « final ».

2.7. Partie 7 – L'historique

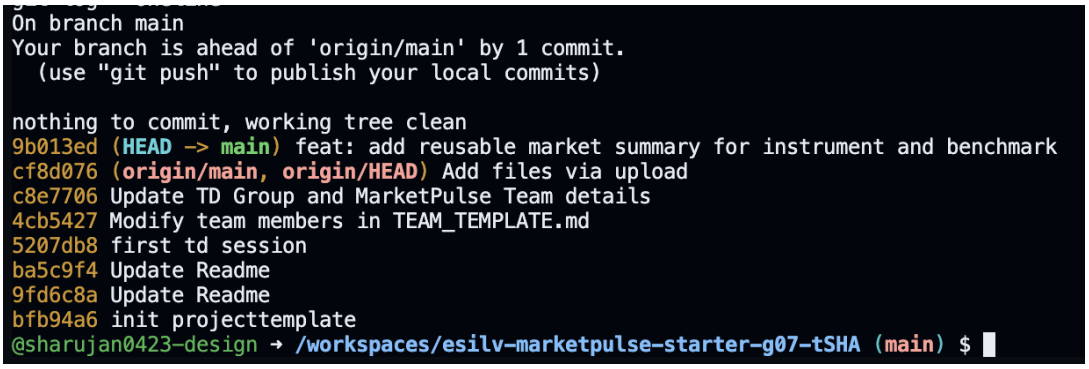


Figure 7 – git status et git log --oneline.

Mon commit `9b013ed` apparaît en haut de l'historique avec son message. `HEAD -> main` indique que c'est le commit actuel de ma branche, et `cf8d076 (origin/main)` est le dernier commit de l'équipe sur GitHub.

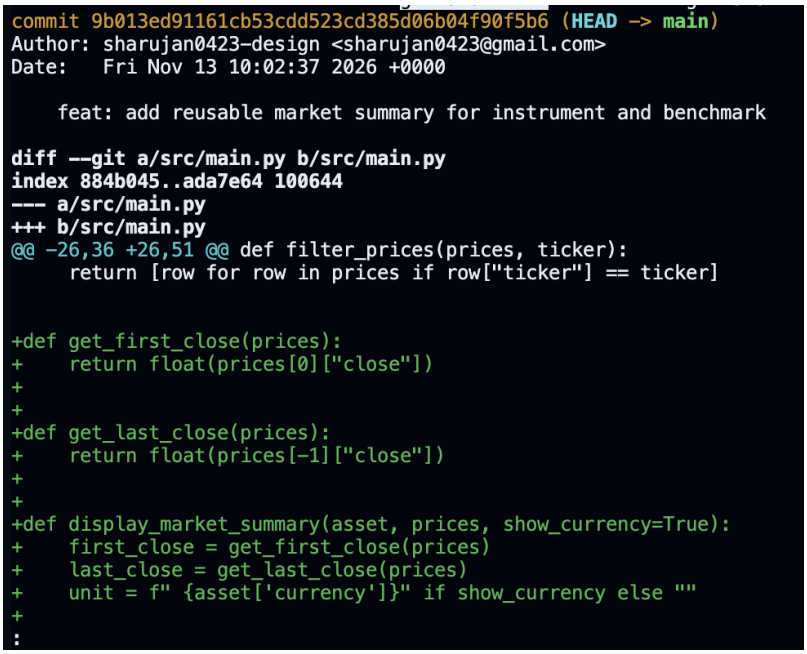


Figure 8 – git show HEAD.

`git show HEAD` donne le détail du commit : un seul fichier modifié (`src/main.py`), l'auteur `sharujan0423-design`, la date (13 novembre 2026, 10:02 UTC) et le diff. On obtient la même chose avec `git show 9b013ed`.

2.8. Partie 8 – Qualité du commit

Mon commit est petit et cohérent : un seul fichier, un seul sujet (le travail du TD02) et un message clair. Je peux expliquer chaque ligne du diff et le refaire. On retrouve mes commits avec `git log --author="sharujan0423-design" --oneline` : c'est l'historique, et pas le fichier TEAM.md, qui montre qui a fait quoi.

2.9. Partie 9 – Local et distant

Le commit n'existe que dans mon Codespace : `git status` indique « ahead of 'origin/main' by 1 commit ». Je n'ai pas fait de push, donc Henri et Adrien ne sont pas impactés. Le travail avec GitHub (push, Pull Request, review) arrive aux TD05 et TD06.

2.10. Partie 10 – Validation

J'ai relancé `git status`, `git log --oneline`, `python src/main.py` et `git show HEAD` (figures 6 à 8) : le dépôt est propre, mon commit est dans l'historique et MarketPulse fonctionne. Le CORE du TD03 est validé, sans push ni Pull Request. La capture de la figure 7 sert de preuve `02_git_status_log.png` pour le Checkpoint A.

3. Conclusion

Ce TD m'a surtout appris à ne plus confondre enregistrer un fichier et enregistrer une version. Le réflexe status, diff, add, diff --staged puis commit permet de vérifier ce qu'on enregistre avant de le faire, ce qui m'a évité de commiter des lignes modifiées par erreur. Mon travail du TD02 est maintenant versionné, ce qui sert de base au TD04 sur les branches.